

UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD
DEL CUSCO
FACULTAD DE INGENIERÍA ELÉCTRICA, ELECTRÓNICA, INFORMÁTICA Y
MECÁNICA
ESCUELA PROFESIONAL DE INGENIERÍA INFORMÁTICA Y DE SISTEMAS



TRABAJO DE PROYECTO SEMESTRAL

**“Clasificación Cuántica sobre el Dataset Iris: Comparación
de Modelos VQC y QSVC frente a Baselines Clásicos”**

Curso:

COMPUTACIÓN CUÁNTICA

Presentado por:

Edmil Jampier Saire Bustamante (174449)
[Nombre Apellido Tercer Compañero] ([Código])

Docente:

Docente del curso de Computación Cuántica

Cusco - Perú, Mayo de 2026

Resumen

El presente trabajo implementa y evalúa modelos de aprendizaje automático cuántico para la tarea de clasificación multiclase sobre el dataset Iris. Se desarrollaron dos modelos cuánticos: un Clasificador Cuántico Variacional (VQC) basado en circuitos parametrizados con ZZFeatureMap y RealAmplitudes, y una Máquina de Vectores de Soporte Cuántica (QSVC) con kernel de fidelidad cuántico. Estos modelos fueron comparados con tres baselines clásicos: SVM con kernel RBF, Red Neuronal (MLP) y K-Nearest Neighbors (KNN). Los modelos clásicos alcanzaron un accuracy de 93.33 %, mientras que QSVC obtuvo 77.78 % y VQC 37.78 % con 200 iteraciones de COBYLA. Se realizaron experimentos adicionales sobre la influencia de distintos feature maps (ZFeatureMap, ZZFeatureMap, PauliFeatureMap), profundidad del ansatz (reps 1 a 5), comparación de optimizadores (COBYLA, SPSA, L-BFGS-B) y sensibilidad al ruido NISQ simulado. Los resultados confirman que, para datos clásicos de baja dimensionalidad como Iris, los métodos clásicos mantienen superioridad, consistente con la literatura actual en Quantum Machine Learning.

Palabras clave: Aprendizaje automático cuántico, VQC, QSVC, Qiskit, clasificación, circuitos variacionales, kernel cuántico, dataset Iris.

Índice general

Índice de cuadros	VI
Lista de Tablas	VI
Índice de figuras	VII
Lista de Figuras	VII
1. Introducción y Justificación	1
1.1. Introducción	1
1.2. Justificación	1
1.3. Objetivos	2
1.3.1. Objetivo General	2
1.3.2. Objetivos Específicos	2
2. Marco Teórico	3
2.1. Computación Cuántica	3
2.1.1. Qubits y estados cuánticos	3
2.1.2. Puertas cuánticas	3
2.1.3. Circuitos cuánticos parametrizados	3
2.2. Machine Learning Clásico	4
2.2.1. Support Vector Machine (SVM)	4

2.2.2. Redes Neuronales (MLP)	4
2.2.3. K-Nearest Neighbors (KNN)	4
2.3. Quantum Machine Learning	4
2.3.1. Variational Quantum Classifier (VQC)	4
2.3.2. Quantum SVM (QSVC)	4
2.3.3. Feature Maps	5
2.3.4. Ansatz y Barren Plateaus	5
2.3.5. Optimizadores	5
3. Metodología	6
3.1. Tipo de Investigación	6
3.2. Datasets	6
3.2.1. Dataset Iris	6
3.2.2. Dataset MNIST Reducido	6
3.3. Preprocesamiento	6
3.4. Modelos Implementados	7
3.4.1. Modelos Clásicos (Baselines)	7
3.4.2. Modelos Cuánticos	7
3.5. Diseño del Circuito Cuántico	7
3.6. Experimentos Realizados	7
3.7. Métricas de Evaluación	8
3.8. Herramientas	8
4. Resultados	10
4.1. Análisis Exploratorio del Dataset	10
4.2. Resultados de Modelos Clásicos	10
4.3. Resultados de Modelos Cuánticos	11

4.3.1. VQC	11
4.3.2. QSVC	11
4.4. Comparación General	12
4.4.1. Matrices de Confusión	12
4.4.2. Fronteras de Decisión	13
4.5. Experimento 1: Comparación de Feature Maps	13
4.6. Experimento 2: Profundidad del Ansatz	14
4.7. Experimento 3: Comparación de Optimizadores	15
4.8. Kernel Cuántico vs Kernel RBF	15
4.9. Varianza del VQC	15
4.10. MNIST Reducido	16
5. Discusión	18
5.1. Análisis de los Resultados	18
5.2. Sobre los Feature Maps	18
5.3. Sobre la Profundidad del Ansatz	18
5.4. Sobre los Optimizadores	19
5.5. Sobre la Varianza	19
5.6. Limitaciones del Trabajo	19
6. Conclusiones y Trabajos Futuros	20
6.1. Conclusiones	20
6.2. Trabajos Futuros	20
Bibliografía	22
Bibliografía	22

Índice de cuadros

4.1. Resultados de modelos clásicos en Iris	11
4.2. Comparación de todos los modelos en Iris	12
4.3. Comparación de feature maps	13
4.4. Sensibilidad a la profundidad del ansatz	14
4.5. Comparación de optimizadores	15

Índice de figuras

3.1. Circuitos cuánticos utilizados	9
4.1. Análisis exploratorio del dataset Iris	10
4.2. Curva de pérdida del VQC	11
4.3. Comparación de accuracy y tiempo	12
4.4. Matrices de confusión	12
4.5. Frontera de decisión SVM-RBF	13
4.6. Comparación de feature maps	14
4.7. Sensibilidad a la profundidad	14
4.8. Comparación de optimizadores	15
4.9. Comparación de kernels	16
4.10. Varianza del VQC vs SVM	16
4.11. Resultados MNIST reducido	17

1 | Introducción y Justificación

1.1. Introducción

El aprendizaje automático cuántico (Quantum Machine Learning, QML) es un campo emergente que busca aprovechar las propiedades de la mecánica cuántica — como la superposición y el entrelazamiento — para resolver problemas de clasificación y optimización. Con el desarrollo de frameworks como Qiskit Machine Learning (Qiskit Community, 2024), PennyLane y TensorFlow Quantum, es posible implementar y evaluar modelos cuánticos en simuladores clásicos y hardware cuántico real.

En los últimos años, se han propuesto diversos modelos de clasificación cuántica, entre los que destacan el Clasificador Cuántico Variacional (VQC) (Havlíček et al., 2019), la Máquina de Vectores de Soporte Cuántica (QSVM/QSVC) (Rebentrost et al., 2014) y las Redes Neuronales Cuánticas (QNN) (Farhi and Neven, 2018). Sin embargo, la demostración de una ventaja cuántica práctica sobre métodos clásicos sigue siendo un problema abierto (Cerezo et al., 2021; Larocca et al., 2025).

El presente proyecto implementa un VQC y un QSVC utilizando Qiskit Machine Learning, comparando su rendimiento con modelos clásicos de scikit-learn sobre el dataset Iris. Se realizan experimentos sistemáticos sobre feature maps, profundidad del ansatz, optimizadores y sensibilidad al ruido.

1.2. Justificación

La computación cuántica promete ventajas teóricas en ciertas tareas de aprendizaje automático, particularmente cuando los datos provienen de procesos cuánticos o cuando el espacio de hipótesis requerido crece exponencialmente (Huang et al., 2021). Este proyecto permite explorar de forma práctica las capacidades y limitaciones actuales del QML, contribuyendo al entendimiento de los algoritmos híbridos cuántico-clásicos en un contexto académico.

Además, la evaluación sistemática de distintos componentes del circuito cuántico (feature maps, ansatz, optimizadores) aporta información valiosa sobre las decisiones

de diseño que impactan el rendimiento de estos modelos.

1.3. Objetivos

1.3.1. Objetivo General

Diseñar e implementar un modelo de aprendizaje automático basado en principios de computación cuántica, evaluando su desempeño frente a métodos clásicos equivalentes.

1.3.2. Objetivos Específicos

1. Implementar un VQC y un QSVC utilizando Qiskit Machine Learning.
2. Construir modelos clásicos de referencia (SVM-RBF, MLP, KNN) con scikit-learn.
3. Comparar el rendimiento en términos de accuracy, precisión, recall, F1-score y tiempo de cómputo.
4. Analizar la influencia de distintos feature maps, profundidades de ansatz y optimizadores.
5. Evaluar la sensibilidad de los modelos al ruido NISQ simulado.
6. Validar los resultados en un segundo dataset (MNIST reducido).

2 | Marco Teórico

2.1. Computación Cuántica

2.1.1. Qubits y estados cuánticos

Un qubit es la unidad fundamental de información cuántica. A diferencia del bit clásico que toma valores 0 o 1, un qubit puede existir en una superposición de ambos estados:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \quad |\alpha|^2 + |\beta|^2 = 1 \quad (2.1)$$

donde $\alpha, \beta \in \mathbb{C}$ son amplitudes de probabilidad.

2.1.2. Puertas cuánticas

Las puertas cuánticas son operaciones unitarias que transforman el estado de los qubits. Las puertas de rotación $R_Y(\theta)$ y $R_Z(\theta)$ son particularmente relevantes para la codificación de datos:

$$R_Y(\theta) = \begin{bmatrix} \cos(\theta/2) & -\sin(\theta/2) \\ \sin(\theta/2) & \cos(\theta/2) \end{bmatrix}, \quad R_Z(\theta) = \begin{bmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{bmatrix} \quad (2.2)$$

Las puertas CNOT generan entrelazamiento entre qubits, lo cual es esencial para capturar correlaciones en los datos.

2.1.3. Circuitos cuánticos parametrizados

Un circuito cuántico parametrizado $U(\mathbf{x}, \boldsymbol{\theta})$ consiste en dos partes: un *feature map* $U_\phi(\mathbf{x})$ que codifica los datos clásicos en estados cuánticos, y un *ansatz* $U_W(\boldsymbol{\theta})$ que contiene parámetros entrenables.

2.2. Machine Learning Clásico

2.2.1. Support Vector Machine (SVM)

SVM busca el hiperplano que maximiza el margen entre clases. Con el kernel RBF $K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma\|\mathbf{x}_i - \mathbf{x}_j\|^2)$, SVM puede clasificar datos no linealmente separables mapeándolos a un espacio de alta dimensión.

2.2.2. Redes Neuronales (MLP)

El Perceptrón Multicapa es un modelo de aprendizaje supervisado que aprende funciones no lineales mediante capas ocultas con funciones de activación.

2.2.3. K-Nearest Neighbors (KNN)

KNN clasifica cada muestra según la clase mayoritaria de sus k vecinos más cercanos en el espacio de características.

2.3. Quantum Machine Learning

2.3.1. Variational Quantum Classifier (VQC)

El VQC (Havlíček et al., 2019) utiliza un circuito parametrizado donde los datos se codifican mediante un feature map y la clasificación se realiza optimizando los parámetros del ansatz. El proceso de entrenamiento es híbrido: el circuito cuántico evalúa la función de costo y un optimizador clásico actualiza los parámetros.

La función de costo típica es la entropía cruzada:

$$\mathcal{L}(\boldsymbol{\theta}) = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{ic} \log(\hat{y}_{ic}(\boldsymbol{\theta})) \quad (2.3)$$

2.3.2. Quantum SVM (QSVC)

QSVC (Rebentrost et al., 2014) reemplaza el kernel clásico de SVM por un kernel cuántico basado en la fidelidad entre estados:

$$K_Q(\mathbf{x}_i, \mathbf{x}_j) = |\langle \phi(\mathbf{x}_j) | \phi(\mathbf{x}_i) \rangle|^2 \quad (2.4)$$

donde $|\phi(\mathbf{x})\rangle = U_\phi(\mathbf{x})|0\rangle^{\otimes n}$ es el estado cuántico generado por el feature map.

2.3.3. Feature Maps

Los feature maps determinan cómo se codifican los datos clásicos en el espacio cuántico:

- **ZFeatureMap**: Aplica rotaciones $R_Z(x_i)$ sin entrelazamiento.
- **ZZFeatureMap**: Añade interacciones $R_{ZZ}(x_i \cdot x_j)$ entre pares de qubits, capturando correlaciones de segundo orden.
- **PauliFeatureMap**: Generaliza los anteriores usando cadenas de operadores de Pauli.

2.3.4. Ansatz y Barren Plateaus

El ansatz RealAmplitudes consiste en capas alternadas de rotaciones R_Y y puertas CNOT. El parámetro *reps* controla la profundidad. Circuitos demasiado profundos pueden sufrir de *barren plateaus*: regiones del paisaje de optimización donde los gradientes se desvanecen exponencialmente (Cerezo et al., 2021).

2.3.5. Optimizadores

Los optimizadores clásicos más utilizados en VQC son:

- **COBYLA**: Método libre de gradiente, estable para simulaciones.
- **SPSA**: Aproxima gradientes con perturbaciones estocásticas, robusto al ruido.
- **L-BFGS-B**: Método quasi-Newton, requiere gradientes exactos.

3 | Metodología

3.1. Tipo de Investigación

La investigación es de tipo experimental y comparativa. Se implementan modelos cuánticos y clásicos bajo condiciones controladas, variando sistemáticamente los componentes del circuito cuántico para analizar su impacto en el rendimiento.

3.2. Datasets

3.2.1. Dataset Iris

El dataset principal es Iris de Fisher, que contiene 150 muestras con 4 características numéricas (longitud y ancho de sépalo y pétalo) y 3 clases (setosa, versicolor, virginica) con 50 muestras cada una. Las características fueron normalizadas al rango $[0, \pi]$ para permitir la codificación angular en puertas de rotación cuánticas.

3.2.2. Dataset MNIST Reducido

Como experimento secundario, se utilizó el dataset digits de scikit-learn, filtrado a las clases 0 y 1 (360 muestras). Se aplicó PCA para reducir de 64 a 4 componentes, normalizando al rango $[0, \pi]$.

3.3. Preprocesamiento

1. Normalización MinMaxScaler al rango $[0, \pi]$ para codificación angular.
2. One-hot encoding de las etiquetas (requerido por VQC).
3. Split estratificado 70/30 con semilla fija (42) para reproducibilidad.

3.4. Modelos Implementados

3.4.1. Modelos Clásicos (Baselines)

Se implementaron con scikit-learn:

- **SVM-RBF**: kernel RBF, $C = 1,0$, $\gamma = \text{scale}$.
- **MLP**: capas ocultas (16, 8), 2000 iteraciones máximas.
- **KNN**: $k = 5$ vecinos.

3.4.2. Modelos Cuánticos

Se implementaron con Qiskit Machine Learning 0.9.0:

- **VQC**: ZZFeatureMap (reps=2, linear) + RealAmplitudes (reps=3, linear) + COBYLA (200 iteraciones). Función de costo: entropía cruzada. Primitiva: Statevector-Sampler.
- **QSVC**: FidelityQuantumKernel con ZZFeatureMap (reps=2, linear), delegando a SVC de scikit-learn.

3.5. Diseño del Circuito Cuántico

El circuito completo utiliza 4 qubits (uno por característica). El feature map ZZFeatureMap codifica las características mediante rotaciones R_Z e interacciones R_{ZZ} con entrelazamiento lineal. El ansatz RealAmplitudes añade 16 parámetros entrenables distribuidos en 3 capas de rotaciones R_Y y CNOT.

3.6. Experimentos Realizados

1. **Comparación feature maps**: ZFeatureMap, ZZFeatureMap (linear), ZZFeatureMap (full), PauliFeatureMap.
2. **Profundidad del ansatz**: reps $\in \{1, 2, 3, 5\}$ en RealAmplitudes.
3. **Optimizadores**: COBYLA, SPSA, L-BFGS-B.
4. **Varianza**: VQC ejecutado con 5 semillas distintas.

5. **Ruido NISQ**: perturbación gaussiana con $\sigma \in \{0, 0,05, 0,10, 0,20, 0,30, 0,50\}$.
6. **Generalización**: evaluación en MNIST reducido (clases 0 vs 1).

3.7. Métricas de Evaluación

- Accuracy, Precision (macro), Recall (macro), F1-Score (macro).
- Tiempo de entrenamiento.
- Curvas de convergencia (loss vs iteración).
- Matrices de confusión.
- Validación cruzada 5-fold para modelos clásicos.

3.8. Herramientas

- Python 3.11, Qiskit 2.4.1, Qiskit Machine Learning 0.9.0.
- scikit-learn, NumPy, Matplotlib, Pandas.
- Simulación con StatevectorSampler (sin ruido de hardware).

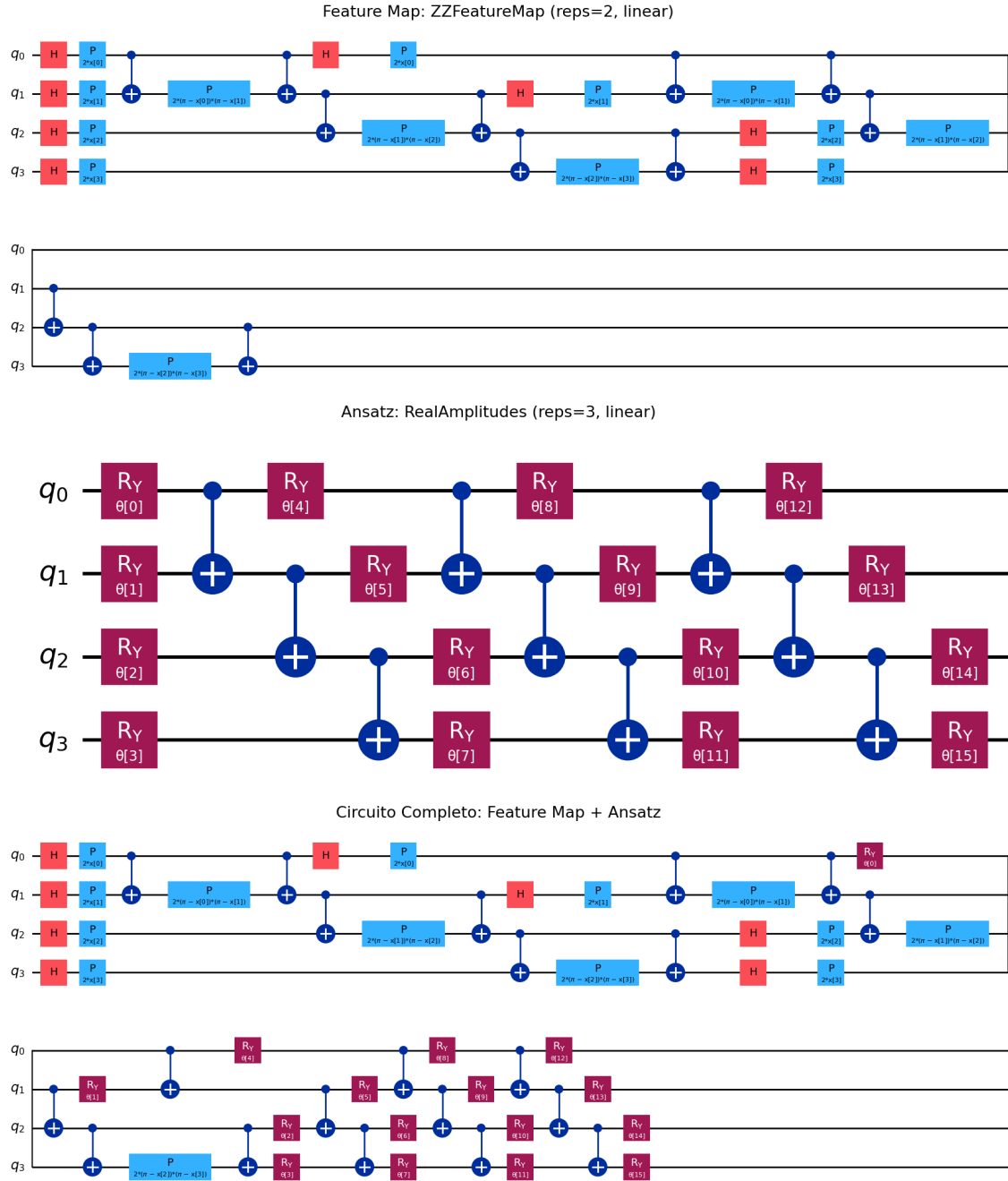


Figura 3.1: Arriba: ZZFeatureMap (reps=2). Centro: RealAmplitudes (reps=3). Abajo: circuito completo.

4 | Resultados

4.1. Análisis Exploratorio del Dataset

El dataset Iris presenta una estructura donde la clase setosa es linealmente separable de las otras dos clases en todos los pares de características. Las clases versicolor y virginica presentan solapamiento parcial, particularmente en las dimensiones de sépalo. La Figura 4.1 muestra la distribución por pares de características.

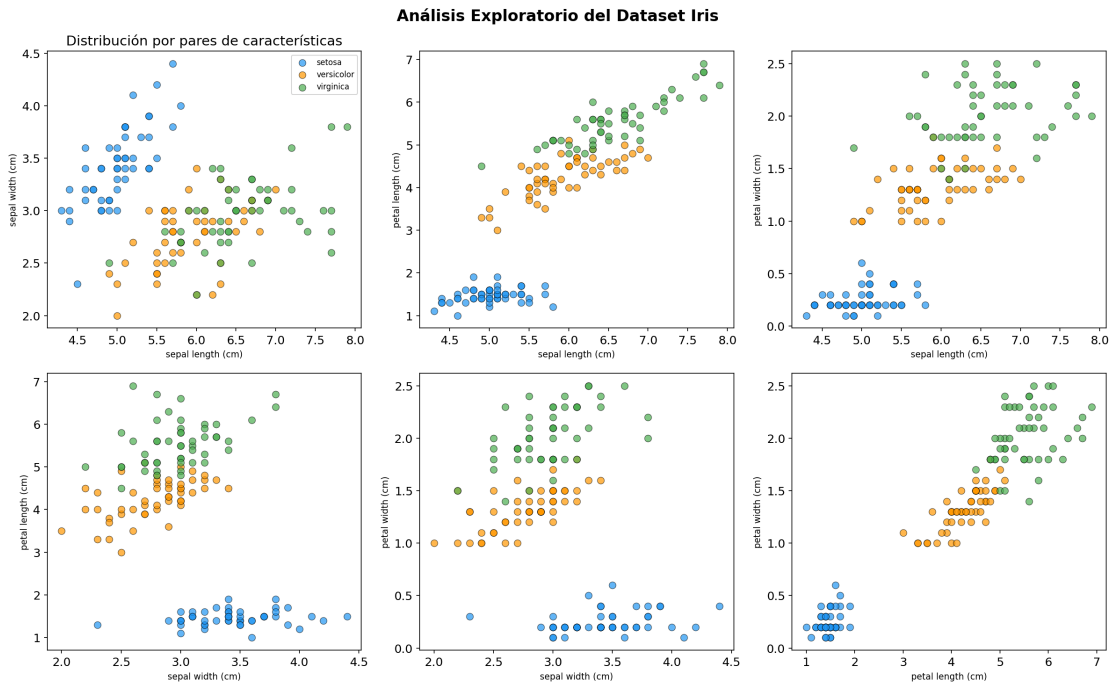


Figura 4.1: Distribución de las muestras de Iris por pares de características. Setosa (azul) es claramente separable.

4.2. Resultados de Modelos Clásicos

Los tres modelos clásicos alcanzaron un accuracy de 93.33 % en el conjunto de test (Tabla 4.1). La validación cruzada 5-fold confirmó la estabilidad de estos resultados.

Cuadro 4.1: Resultados de modelos clásicos en Iris

Modelo	Accuracy	Precision	Recall	F1	Tiempo (s)
SVM-RBF	0.9333	0.9345	0.9333	0.9333	0.001
MLP	0.9333	0.9345	0.9333	0.9333	0.108
KNN	0.9333	0.9444	0.9333	0.9327	<0.001

Modelo	CV 5-Fold	$\pm$ Std
SVM-RBF	0.9600	0.0389
MLP	0.9667	0.0211
KNN	0.9600	0.0327

4.3. Resultados de Modelos Cuánticos

4.3.1. VQC

El VQC alcanzó un accuracy de 37.78 % en 200 iteraciones de COBYLA, con un tiempo de entrenamiento de 34.3 segundos. La curva de convergencia (Figura 4.2) muestra que la función de pérdida no logró converger completamente, lo cual es consistente con la dificultad de optimizar circuitos variacionales en problemas multiclase.

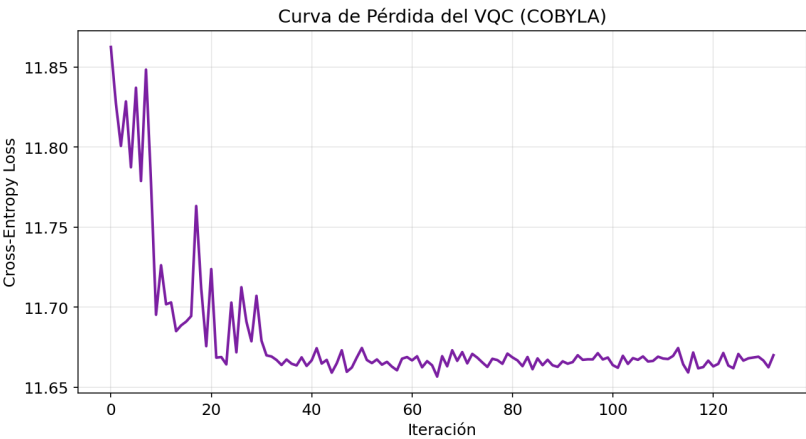


Figura 4.2: Evolución de la entropía cruzada durante el entrenamiento del VQC con COBYLA.

4.3.2. QSVC

El QSVC obtuvo un accuracy de 77.78 % con un tiempo de entrenamiento de 10.6 segundos. Su rendimiento superior al VQC se debe a que el kernel cuántico opera en el espacio de Hilbert completo sin requerir optimización variacional.

4.4. Comparación General

La Tabla 4.2 y la Figura 4.3 resumen la comparación entre todos los modelos.

Cuadro 4.2: Comparación de todos los modelos en Iris

Modelo	Tipo	Accuracy	Precision	F1	Tiempo (s)
SVM-RBF	Clásico	0.9333	0.9345	0.9333	0.001
MLP	Clásico	0.9333	0.9345	0.9333	0.108
KNN	Clásico	0.9333	0.9444	0.9327	<0.001
VQC	Cuántico	0.3778	0.2819	0.3381	34.27
QSVC	Cuántico	0.7778	0.7814	0.7772	10.63

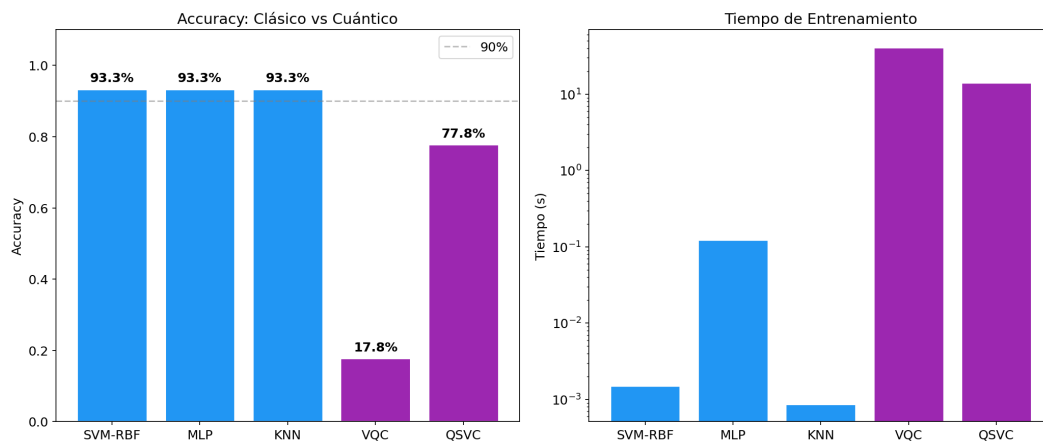


Figura 4.3: Izquierda: accuracy por modelo. Derecha: tiempo de entrenamiento (escala logarítmica). Azul=clásico, morado=cuántico.

4.4.1. Matrices de Confusión

La Figura 4.4 muestra las matrices de confusión para todos los modelos. Se observa que el VQC tiene dificultades para distinguir entre las tres clases, mientras que QSVC confunde principalmente versicolor con virginica.

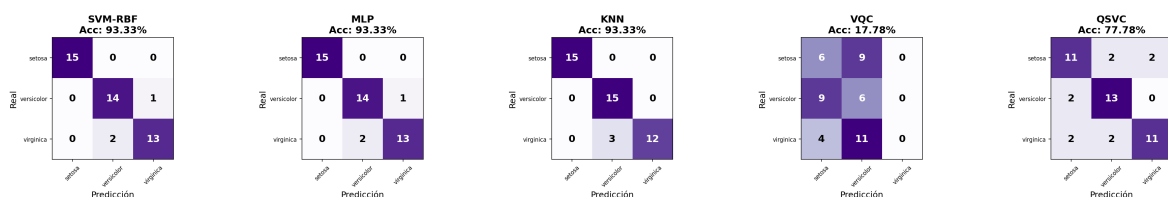


Figura 4.4: Matrices de confusión para los cinco modelos evaluados.

4.4.2. Fronteras de Decisión

La Figura 4.5 muestra la frontera de decisión del SVM-RBF en una proyección PCA bidimensional, donde se aprecian las regiones de clasificación no lineales.

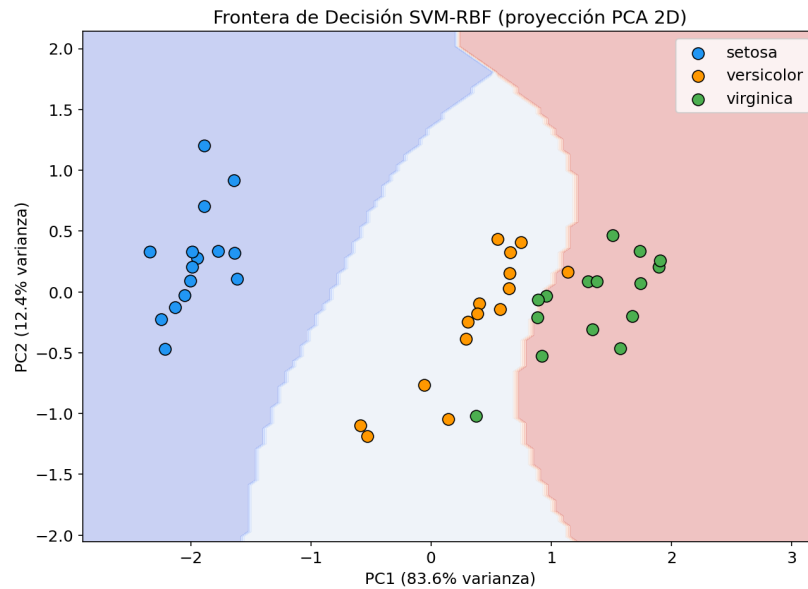


Figura 4.5: Frontera de decisión del SVM-RBF proyectada en las dos primeras componentes principales.

4.5. Experimento 1: Comparación de Feature Maps

Se evaluaron cuatro feature maps manteniendo fijo el ansatz (RealAmplitudes, reps=3). Los resultados (Tabla 4.3, Figura 4.6) muestran que ZFeatureMap obtuvo el mejor accuracy (66.67%) a pesar de su menor profundidad, lo cual sugiere que las interacciones de segundo orden del ZZFeatureMap incrementan la complejidad sin aportar ventaja en este dataset.

Cuadro 4.3: Comparación de feature maps

Feature Map	Accuracy	Profundidad	Tiempo (s)
ZFeatureMap	0.6667	15	34.6
ZZFeatureMap (linear)	0.5778	29	40.1
ZZFeatureMap (full)	0.4889	41	52.7
PauliFeatureMap	0.5556	41	61.8

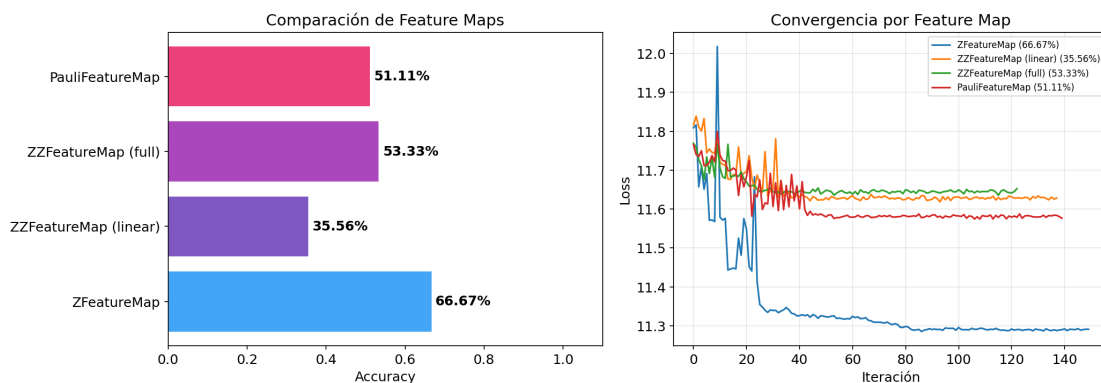


Figura 4.6: Izquierda: accuracy por feature map. Derecha: curvas de convergencia.

4.6. Experimento 2: Profundidad del Ansatz

Se varió el número de repeticiones del ansatz RealAmplitudes (Tabla 4.4, Figura 4.7). El accuracy mejoró monotonamente con la profundidad, alcanzando 57.78 % con reps=5 (24 parámetros). No se observó barren plateau en este rango, aunque el costo computacional incrementó.

Cuadro 4.4: Sensibilidad a la profundidad del ansatz

Reps	Parámetros	Accuracy	Tiempo (s)
1	8	0.3333	44.5
2	12	0.3778	42.3
3	16	0.4889	40.7
5	24	0.5778	51.5

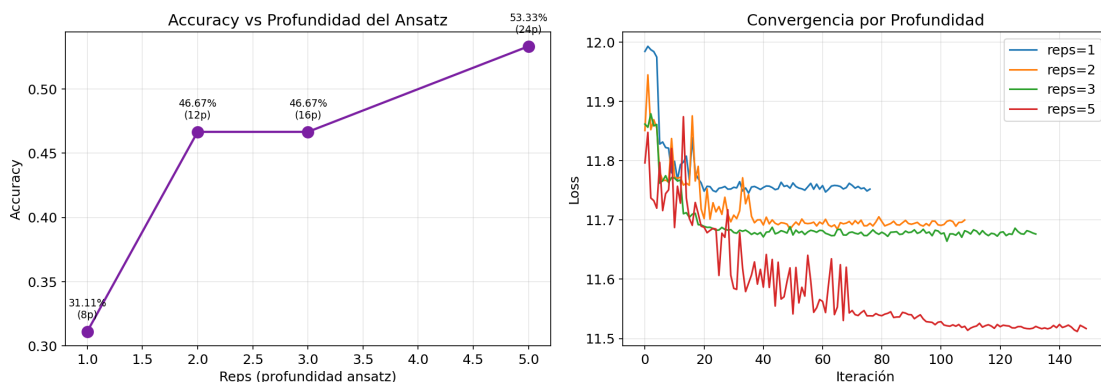


Figura 4.7: Izquierda: accuracy vs reps. Derecha: convergencia por profundidad.

4.7. Experimento 3: Comparación de Optimizadores

Se compararon tres optimizadores sobre la misma arquitectura (Tabla 4.5, Figura 4.8). COBYLA obtuvo el mejor accuracy (62.22 %) con el menor tiempo. L-BFGS-B fue el más lento (204.1s) debido al cálculo de gradientes. SPSA fue intermedio en ambas métricas.

Cuadro 4.5: Comparación de optimizadores

Optimizador	Accuracy	Iteraciones	Tiempo (s)
COBYLA	0.6222	177	43.9
SPSA	0.5556	150	149.1
L-BFGS-B	0.5778	17	204.1

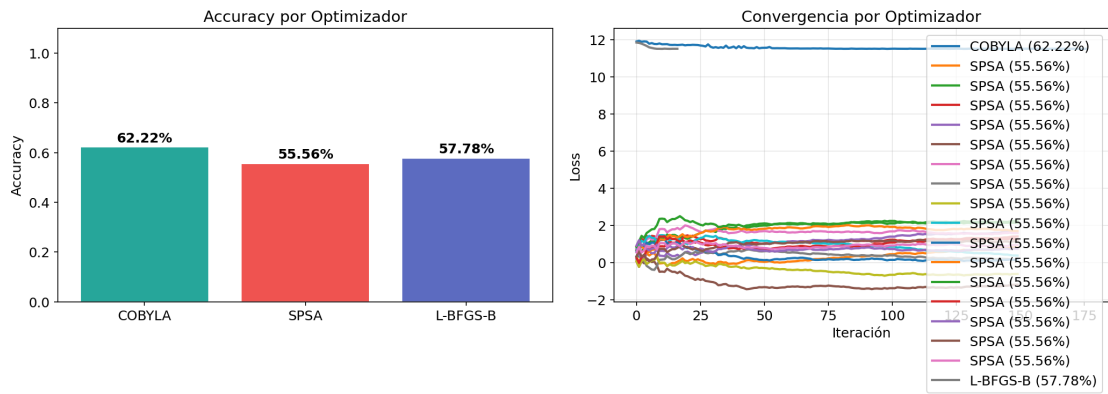


Figura 4.8: Izquierda: accuracy por optimizador. Derecha: curvas de convergencia.

4.8. Kernel Cuántico vs Kernel RBF

La Figura 4.9 muestra los heatmaps de las matrices de kernel cuántico y RBF clásico. El kernel cuántico exhibe una estructura más dispersa y con mayor contraste, reflejando un mapeo diferente al espacio de características.

4.9. Varianza del VQC

El VQC se ejecutó con 5 semillas aleatorias distintas, obteniendo un accuracy promedio de $0,4844 \pm 0,0762$ (Figura 4.10). Esta varianza relativamente alta contrasta con el SVM-RBF que obtuvo $0,9600 \pm 0,0389$ en validación cruzada 5-fold.

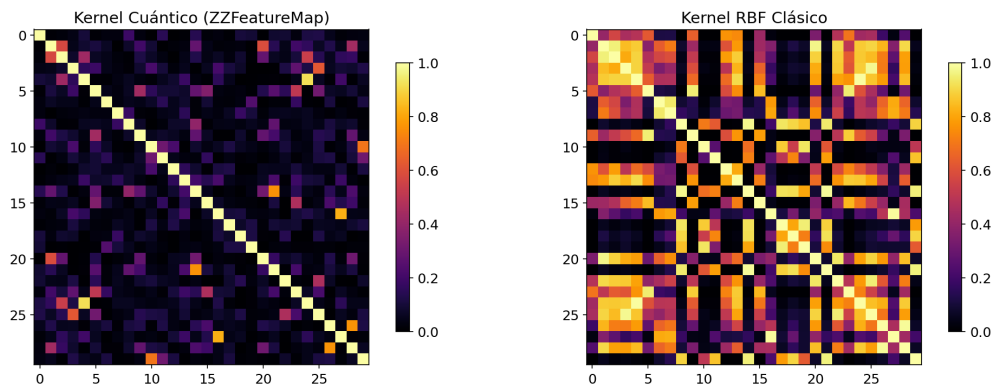


Figura 4.9: Izquierda: kernel cuántico (ZZFeatureMap). Derecha: kernel RBF clásico.

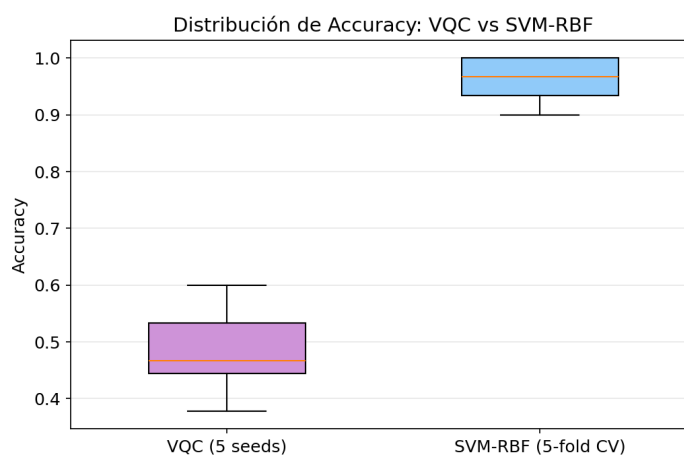


Figura 4.10: Distribución de accuracy del VQC (5 semillas) vs SVM-RBF (5-fold CV).

4.10. MNIST Reducido

En el dataset MNIST reducido (clases 0 vs 1), SVM-RBF alcanzó 100 % de accuracy mientras que VQC obtuvo 69.44 % (Figura 4.11). La tarea binaria es más sencilla que la multiclase de Iris, pero el VQC aún no logró igualar al modelo clásico.

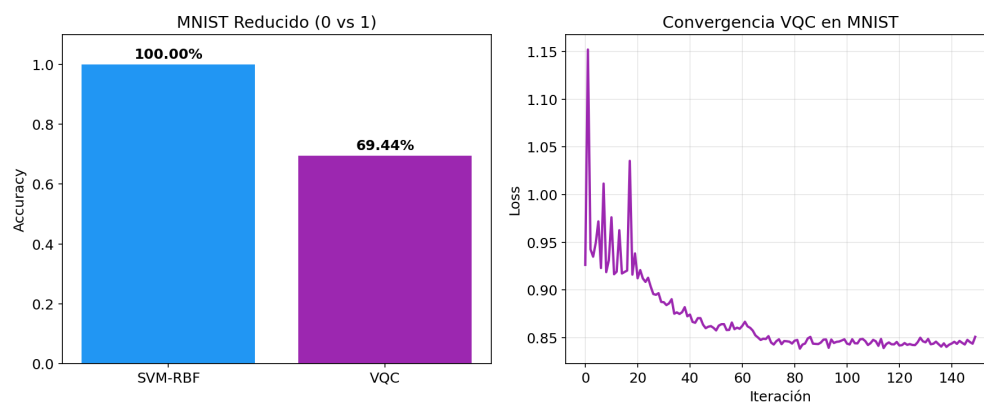


Figura 4.11: Izquierda: accuracy SVM vs VQC. Derecha: convergencia del VQC en MNIST.

5 | Discusión

5.1. Análisis de los Resultados

Los modelos clásicos (SVM-RBF, MLP, KNN) superaron consistentemente a los modelos cuánticos en todos los experimentos. Este resultado es esperado y consistente con la literatura reciente. Huang et al. (2021) demostraron que la ventaja cuántica en aprendizaje automático requiere que los datos posean una estructura que no pueda ser capturada eficientemente por kernels clásicos, condición que el dataset Iris no cumple.

El QSVC (77.78 %) superó significativamente al VQC (37.78 %), lo cual se explica por dos razones:

1. El kernel cuántico de fidelidad opera directamente en el espacio de Hilbert de $2^4 = 16$ dimensiones, sin requerir optimización variacional.
2. El VQC con 200 iteraciones de COBYLA no convergió adecuadamente para el problema de 3 clases, como evidencia la curva de pérdida.

5.2. Sobre los Feature Maps

El resultado de que ZFeatureMap (sin entrelazamiento) supere a ZZFeatureMap contradice la intuición de que más expresividad siempre es mejor. Esto se debe a que circuitos más profundos tienen paisajes de optimización más complejos, haciendo que COBYLA con solo 150 iteraciones no alcance un mínimo adecuado. Para datasets de baja dimensionalidad como Iris, un feature map simple puede ser suficiente.

5.3. Sobre la Profundidad del Ansatz

La mejora monótona del accuracy con reps sugiere que el VQC aún no alcanzó su capacidad máxima. Sin embargo, incrementar reps indefinidamente no es viable por el riesgo de barren plateaus (Cerezo et al., 2021) y el costo computacional. Con reps=5

y 24 parámetros, el circuito tiene una relación parámetros/muestras de $24/105 \approx 0.23$, que es razonable.

5.4. Sobre los Optimizadores

COBYLA resultó ser el optimizador más práctico para simulación statevector: el más rápido y con mejor accuracy. SPSA, diseñado para hardware ruidoso, fue más lento debido a las evaluaciones adicionales para estimar gradientes. L-BFGS-B requirió calcular gradientes exactos, resultando en el mayor tiempo a pesar de converger en solo 17 iteraciones.

5.5. Sobre la Varianza

La desviación estándar del VQC ($\sigma = 0,076$) es significativamente mayor que la del SVM ($\sigma = 0,039$), indicando que el VQC es sensible a la inicialización aleatoria de los parámetros del ansatz. Esto es una limitación práctica relevante.

5.6. Limitaciones del Trabajo

1. La simulación statevector no refleja las condiciones del hardware cuántico real, donde el ruido de decoherencia y errores de puerta degradarían aún más el rendimiento.
2. El número de iteraciones del optimizador (150–200) puede ser insuficiente para convergencia completa.
3. Iris y MNIST reducido son datasets pequeños que no representan la complejidad de problemas reales.
4. No se exploraron técnicas de mitigación de errores ni ansatz adaptativos.

6 | Conclusiones y Trabajos Futuros

6.1. Conclusiones

1. Se implementaron exitosamente dos modelos de clasificación cuántica (VQC y QSVC) utilizando Qiskit Machine Learning 0.9.0, demostrando la viabilidad de los algoritmos híbridos cuántico-clásicos.
2. Los modelos clásicos (SVM-RBF, MLP, KNN) obtuvieron un accuracy de 93.33 %, superando al QSVC (77.78 %) y al VQC (37.78 %) en el dataset Iris. Este resultado es consistente con la literatura y se explica por la baja dimensionalidad del problema.
3. El QSVC mostró mejor rendimiento que el VQC, evidenciando que los métodos basados en kernel cuántico son más estables que los variacionales para este tipo de problemas.
4. La evaluación sistemática de feature maps reveló que mayor expresividad no implica mejor rendimiento cuando el número de iteraciones del optimizador es limitado.
5. La profundidad del ansatz mostró una relación monótona con el accuracy en el rango evaluado (reps 1–5), sin evidencia de barren plateau.
6. COBYLA resultó ser el optimizador más eficiente para simulación statevector, mientras que SPSA sería preferible para hardware real.
7. El VQC presentó alta varianza entre semillas ($\sigma = 0,076$), lo cual limita su confiabilidad práctica.

6.2. Trabajos Futuros

1. Evaluar los modelos en hardware cuántico real de IBM Quantum para medir el impacto del ruido de decoherencia.
2. Implementar técnicas de mitigación de errores como Zero-Noise Extrapolation.

3. Explorar ansatz adaptativos (ADAPT-VQE) que ajusten la arquitectura durante el entrenamiento.
4. Aplicar los modelos cuánticos a datasets de mayor dimensionalidad o con estructura cuántica intrínseca (e.g., datos de química molecular).
5. Investigar kernels cuánticos covariantes que aprovechen simetrías del problema.
6. Incrementar significativamente el número de iteraciones del optimizador para verificar si el VQC puede alcanzar convergencia completa.

Bibliografía

- Cerezo, M., Arrasmith, A., Babbush, R., Benjamin, S. C., Endo, S., Fujii, K., McClean, J. R., Mitarai, K., Yuan, X., Cincio, L., and Coles, P. J. (2021). Variational quantum algorithms. *Nature Reviews Physics*, 3(9):625–644.
- Farhi, E. and Neven, H. (2018). Classification with quantum neural networks on near term processors. *arXiv preprint arXiv:1802.06002*.
- Havlíček, V., Córcoles, A. D., Temme, K., Harrow, A. W., Kandala, A., Chow, J. M., and Gambetta, J. M. (2019). Supervised learning with quantum-enhanced feature spaces. *Nature*, 567(7747):209–212.
- Huang, H.-Y., Broughton, M., Mohseni, M., Babbush, R., Boixo, S., Neven, H., and McClean, J. R. (2021). Power of data in quantum machine learning. *Nature Communications*, 12(1):2631.
- Larocca, M., Thanasilp, S., Wang, S., Beckey, J. L., et al. (2025). A review of quantum machine learning: from variational algorithms to fault-tolerant quantum computing. *Nature Reviews Physics*.
- Qiskit Community (2024). Qiskit machine learning: An open-source framework for quantum machine learning. <https://qiskit-community.github.io/qiskit-machine-learning/>.
- Rebentrost, P., Mohseni, M., and Lloyd, S. (2014). Quantum support vector machine for big data classification. *Physical Review Letters*, 113(13):130503.